

Name:

Muhammad Abdullah Aamir

Internship ID:

ZYNVEX-CERT-1681

Type:

Project

Week:

Week 04

Topic:

Phishing Detection

ZYNVEX SOLUTIONS INTERNSHIP

PHISH URL

Phishing URL and Message Scanner with API and Browser Extension

Resource	Link
Live application	https://phishurl-abdullah.streamlit.app/
Source code (GitHub)	https://github.com/abdullah-aesthetics/PhishURL
Dataset (Kaggle)	https://www.kaggle.com/datasets/hemanthpingali/phishing-url

1. Project Overview:

Phishing links are one of the most common ways attackers steal passwords, card details and money. A link in an email, SMS or WhatsApp message imitates a trusted service, and many users cannot tell it apart from a genuine one. Blocklists help, but new phishing sites appear faster than lists can be updated.

PhishURL is a web application that analyses the **text of a URL alone** and predicts whether it is legitimate or phishing. It never opens or visits the link, so scanning a malicious URL cannot harm the user. For each link it returns a verdict, a risk score from 0 to 100% and plain-English reasons for the decision. The same models power three interfaces: a Streamlit dashboard (URL scanner, SMS/message scanner and batch scanner), a REST API, and a Chrome extension that shows a warning badge while the user browses.

PhishURL is a companion to the earlier PhishGuard project, which detects phishing from the body text of emails. Together they cover the two most common attack carriers: email text and links.

2. Objectives:

1. Build and compare machine-learning classifiers that detect phishing URLs from the URL text only.
2. Make every decision explainable through a risk score and human-readable reasons.
3. Evaluate honestly with a stratified hold-out test set and cross-validation, and report weaknesses as well as strengths.
4. Deliver the system as a deployed dashboard, a REST API and a browser extension, not just a notebook.
5. Provide a tested, documented and version-controlled codebase.

3. Features and Technologies Used:

3.1 Features:-

- **URL scanner:** verdict, ML probability, rule score, animated risk gauge and reasons for each URL.
- **Message scanner:** paste an SMS or WhatsApp message; links are extracted, each is scored, and scam wording is listed.

- **Batch scanner:** scan many URLs from pasted text or a CSV, with summary cards, a risk-level chart and a downloadable results file.
- **Model comparison:** Random Forest, character n-gram Logistic Regression and a Hybrid model, with a selectable model in the sidebar.
- **Performance dashboard:** accuracy, precision, recall, F1, ROC-AUC, feature importances and confusion matrices.
- **REST API:** a FastAPI service with a /scan endpoint and automatic interactive documentation.
- **Chrome extension (PhishURL Shield):** a toolbar badge (OK, ! or !!) for each page and a popup with the verdict and reasons.
- **Session history** of scans and automated unit tests.

3.2 Technologies:-

Technology	Purpose
Python 3	Main programming language
scikit-learn	Feature pipelines, TF-IDF, Random Forest, Logistic Regression, evaluation
pandas, NumPy	Dataset loading and feature tables
Streamlit and Altair	Interactive dashboard and charts
FastAPI and Uvicorn	REST API used by the browser extension
JavaScript (Manifest V3)	Chrome extension (background service worker and popup)
joblib	Saving and loading the trained models
pytest	Automated tests
Git, GitHub, Streamlit Community Cloud	Version control and public deployment

4. System Workflow and Architecture:

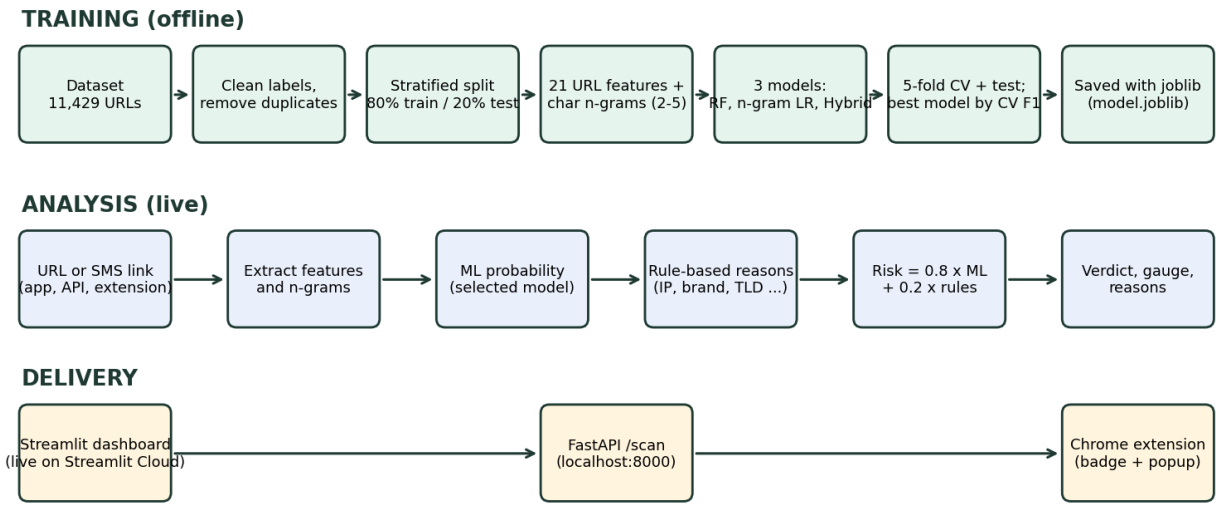


Figure 1: PhishURL architecture: offline training, live analysis and delivery channels

4.1 Training phase (offline):-

1. **Dataset:** two Kaggle files were combined into 11,429 unique labelled URLs.
2. **Cleaning:** labels were converted to 1 (phishing) and 0 (legitimate) and duplicate URLs were removed.
3. **Split:** a stratified 80/20 split produced 9,143 training and 2,286 test URLs.
4. **Features:** 21 lexical features are computed from the URL text (length, IP host, subdomains, hyphens, digit ratio, entropy of the host, sensitive words, brand mismatch, suspicious top-level domain, HTTPS, shortener, port, punycode and others). In addition, TF-IDF over character n-grams (2 to 5 characters) captures patterns inside the URL string.
5. **Models:** Random Forest on the 21 features; Logistic Regression on character n-grams; and a Hybrid Logistic Regression on n-grams plus the scaled 21 features.
6. **Evaluation and selection:** 5-fold stratified cross-validation on the training set, then one evaluation on the test set. The default model is chosen by cross-validated F1, never by test score.

4.2 Analysis phase (live):-

When a user submits a URL, the selected model returns a phishing probability and rule-based checks list readable reasons (for example, a raw IP address, a brand name on a domain that is not the brand, a high-abuse top-level domain, no HTTPS, or several sensitive words). The two are combined:

Risk score = 0.8 x ML probability + 0.2 x rule score

The rule score is the sum of reason weights (high = 3, medium = 2, low = 1) divided by 8 and capped at 1. A risk below 0.35 is Low, below 0.65 is Medium and anything higher is High. The verdict (likely phishing or likely legitimate) follows the ML probability at a 0.5 threshold. The URL is analysed as text only and is never opened.

5. Installation and Setup:

Requirements: Python 3.10 or newer, Git and (optionally) Google Chrome for the extension. Commands are for Windows PowerShell.

```
git clone https://github.com/abdullah-aesthetics/PhishURL.git
cd PhishURL
py -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python train.py            # trains on data/urls.csv and saves models/
python -m pytest -q        # runs the tests
python -m streamlit run app.py   # dashboard at http://localhost:8501
```

API: in a second terminal run `python -m uvicorn api:app --port 8000`; interactive documentation is then available at http://localhost:8000/docs.

Browser extension: open chrome://extensions, enable Developer mode, choose Load unpacked and select the extension folder. Keep the API running while browsing.

Retraining on other data: save a CSV with columns url and label to data/urls.csv and run python train.py. A trained model is already included in the repository, and the deployed app is available online without any installation.

6. Usage and Screenshots:

6.1 Live deployment and URL scanner:-

The dashboard is deployed publicly at https://phishurl-abdullah.streamlit.app/. Figure 2 shows the live application scanning a fake PayPal login link.

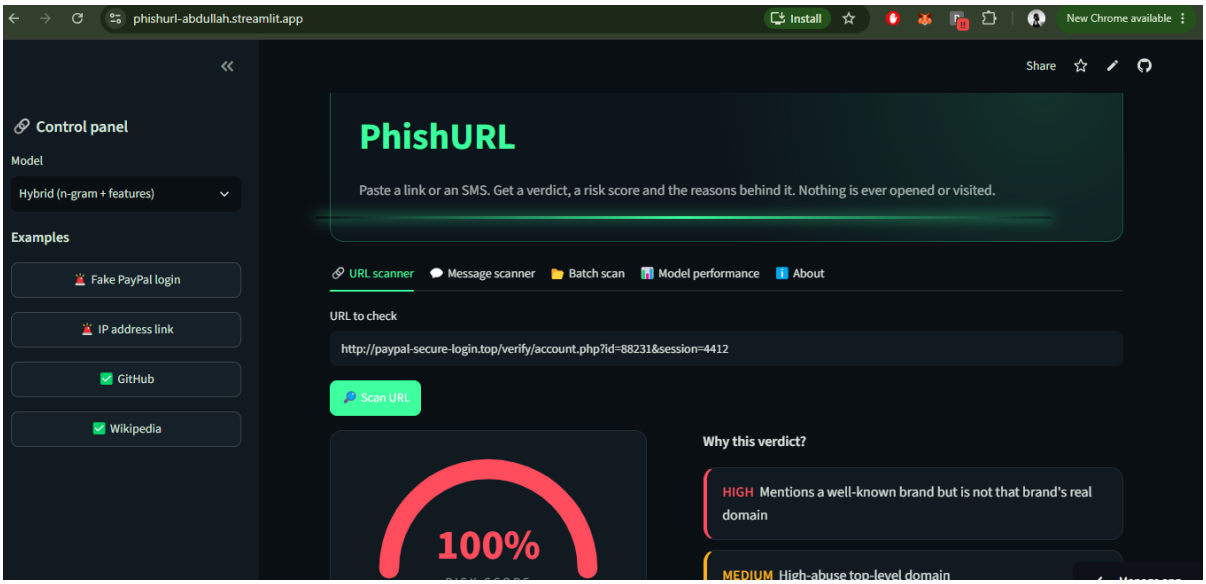


Figure 2: The deployed application on Streamlit Community Cloud (phishurl-abdullah.streamlit.app)

The home page offers four example links in the sidebar and a model selector. The user pastes a URL and presses Scan URL.

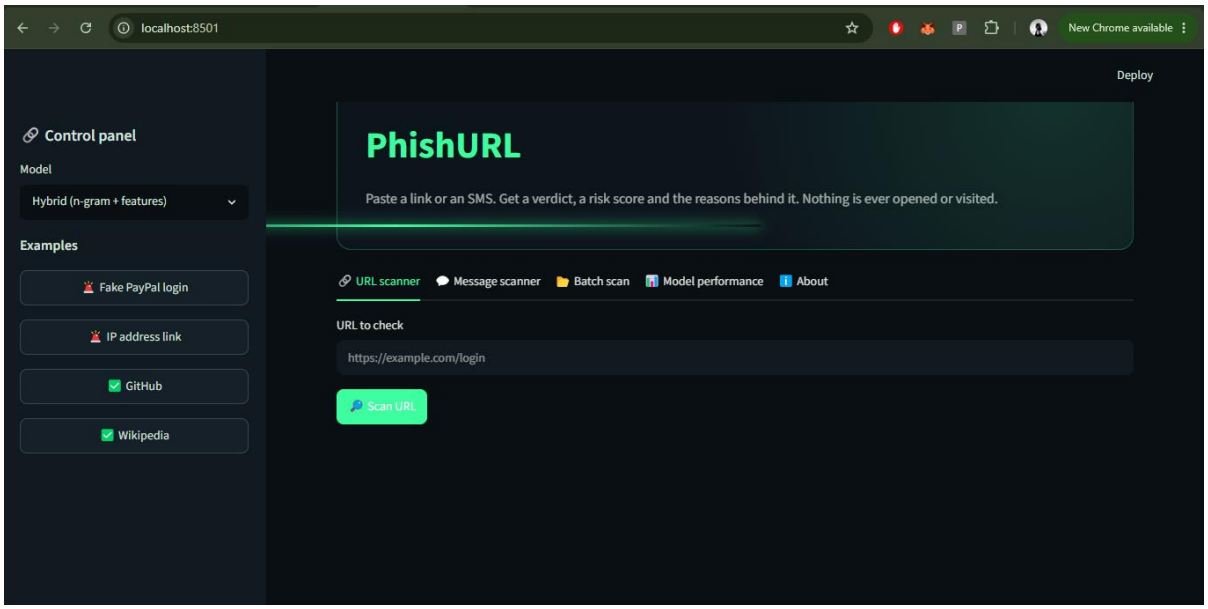


Figure 3: Dashboard home page with the URL scanner, the animated scanner bar and the five tabs

A phishing link is scored 100% risk. The reasons explain the verdict: a well-known brand on a domain that is not the brand's, a high-abuse top-level domain, no HTTPS and several sensitive words. A legitimate link scores low and shows no warnings.

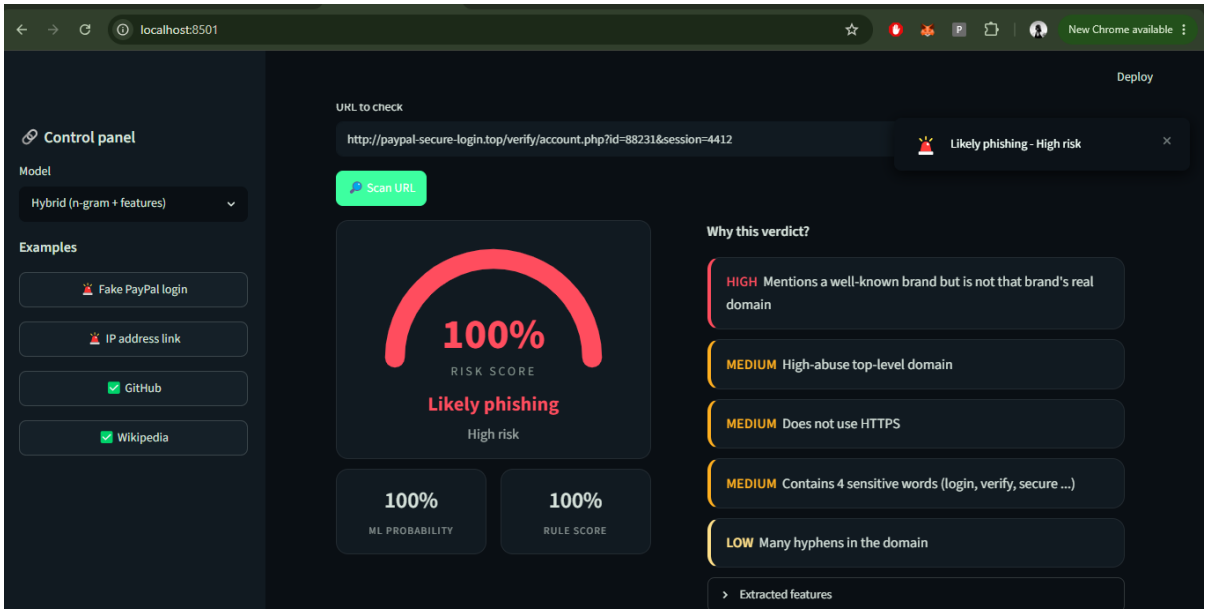


Figure 4: A fake PayPal login URL classified as likely phishing with 100% risk and its reasons

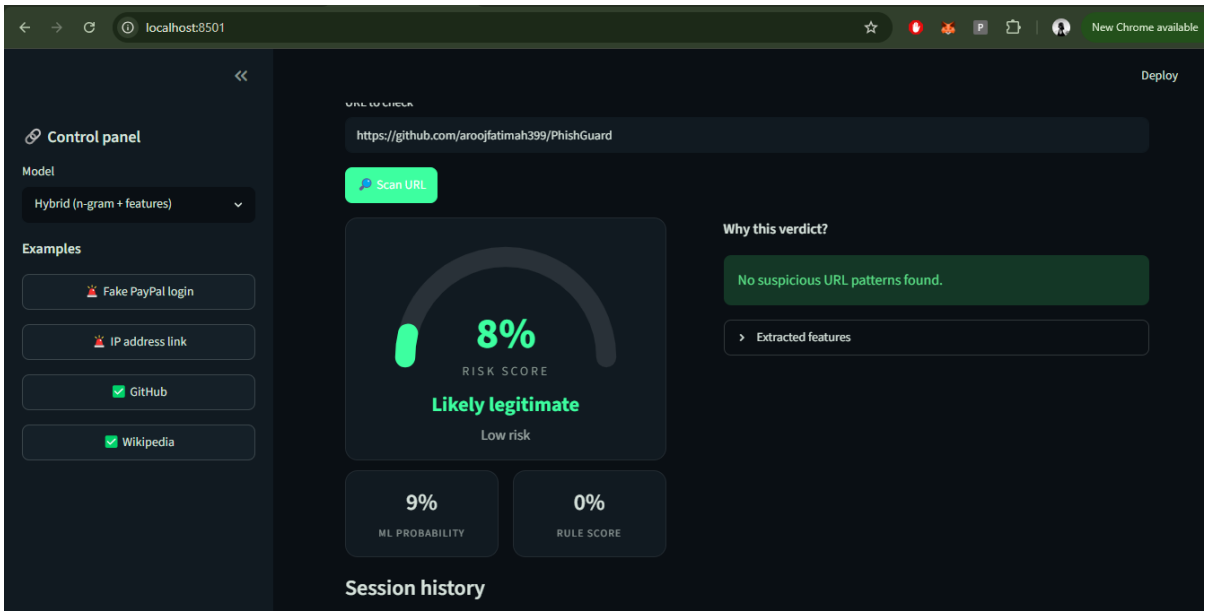


Figure 5: A legitimate GitHub URL classified as likely legitimate with 8% risk

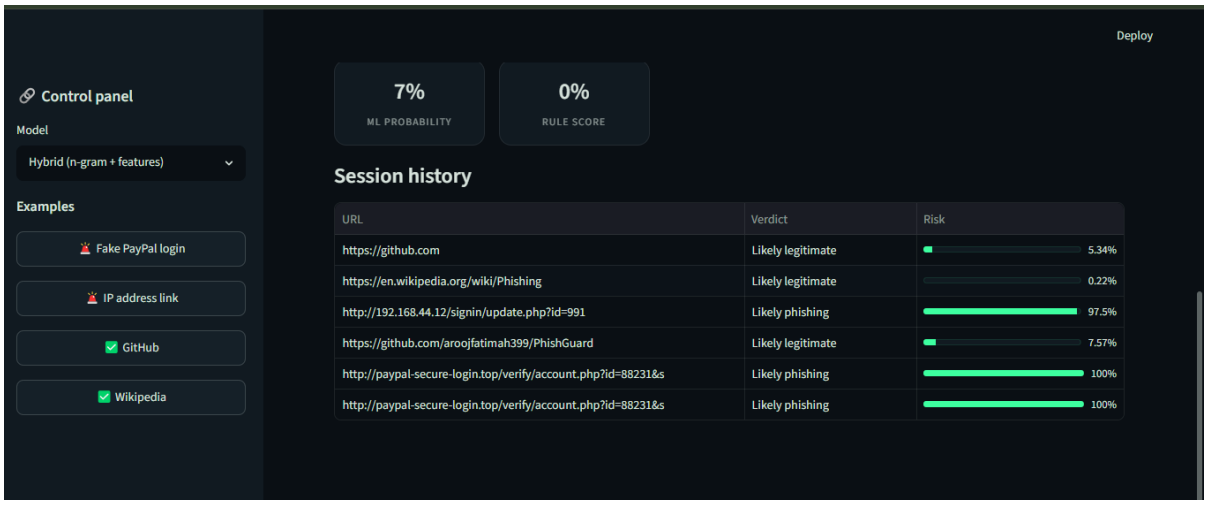


Figure 6: Session history listing every scan with its verdict and risk bar

6.2 Message scanner:-

Pasting an SMS or WhatsApp message finds every link inside it, lists scam wording such as parcel and customs, and scores each link separately.

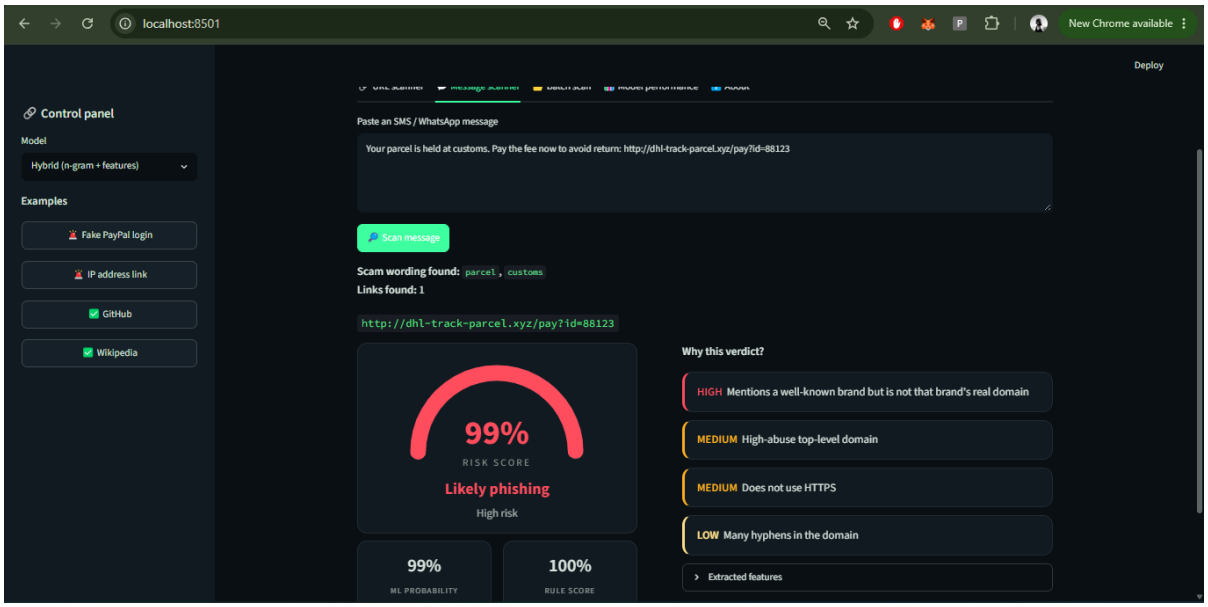


Figure 7: A parcel-delivery scam message: scam wording found and its link scored 99% risk

6.3 Batch scanner:-

The Batch scan tab scores many URLs at once and shows a risk-level chart, a table with risk bars and a reason column, and a CSV download. In this test two of the legitimate links were wrongly flagged; this is discussed in Section 8.

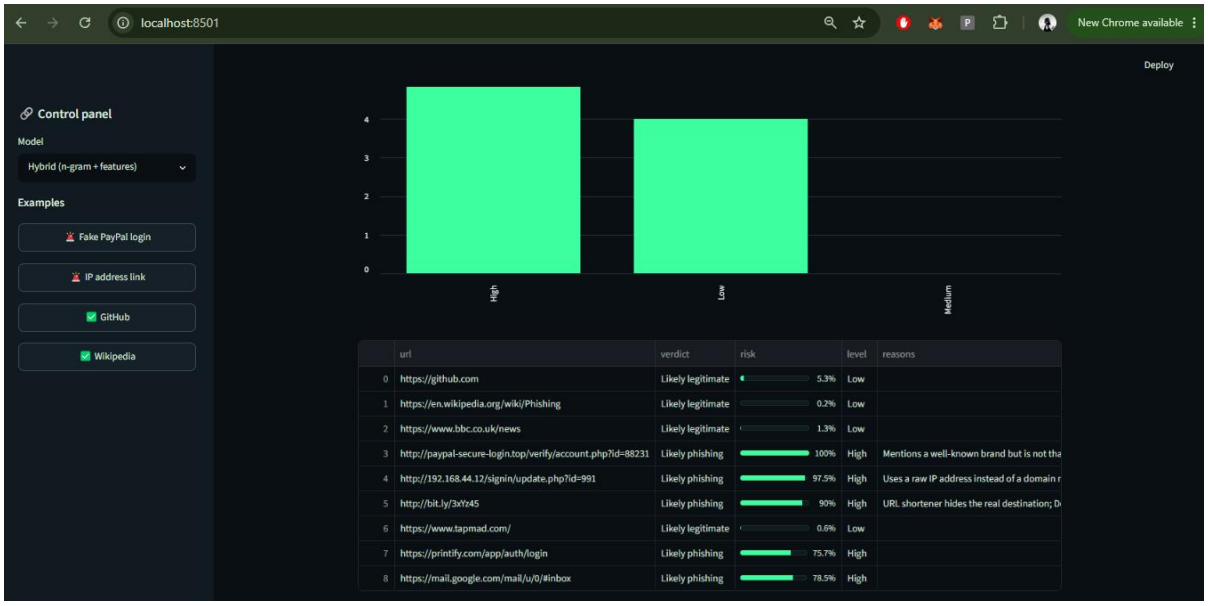


Figure 8: Batch scan of nine URLs with the risk-level chart and the results table

6.4 REST API:-

The same model is exposed through a FastAPI service. FastAPI generates interactive documentation automatically, so the endpoints can be tested from the browser.

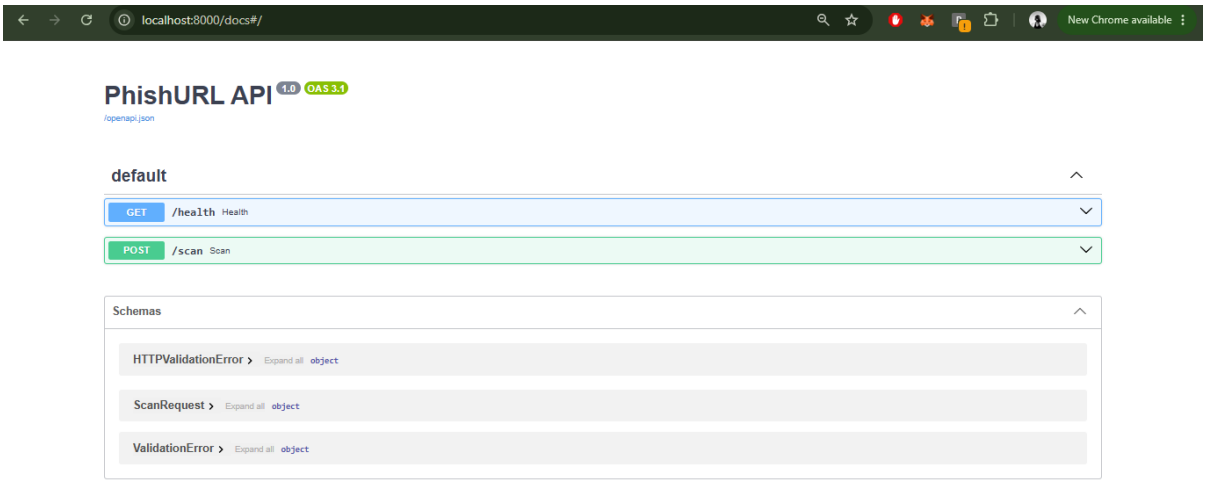


Figure 9: PhishURL API documentation page with the /health and /scan endpoints

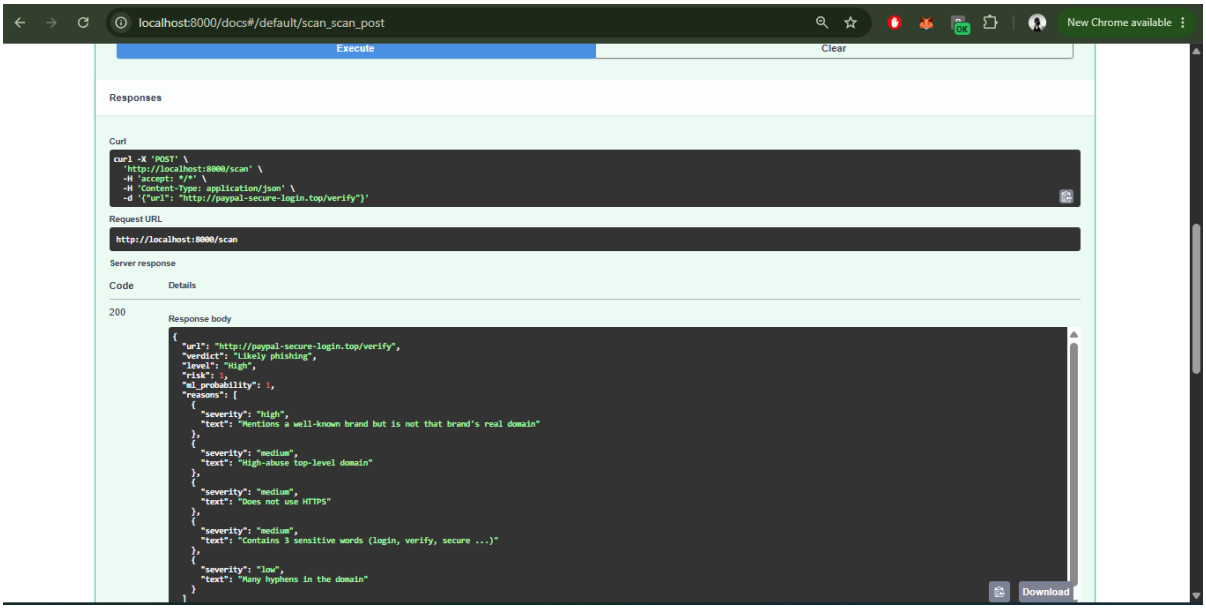


Figure 10: A POST /scan request and its JSON response: verdict, risk level, probability and reasons

6.5 Chrome extension:-

The PhishURL Shield extension sends the address of the current page to the local API and shows a badge on the toolbar icon: green OK for low risk, amber ! for medium and red !! for high. Clicking the icon opens a popup with the verdict, a risk bar and the reasons. Only the URL is sent, never the page content.

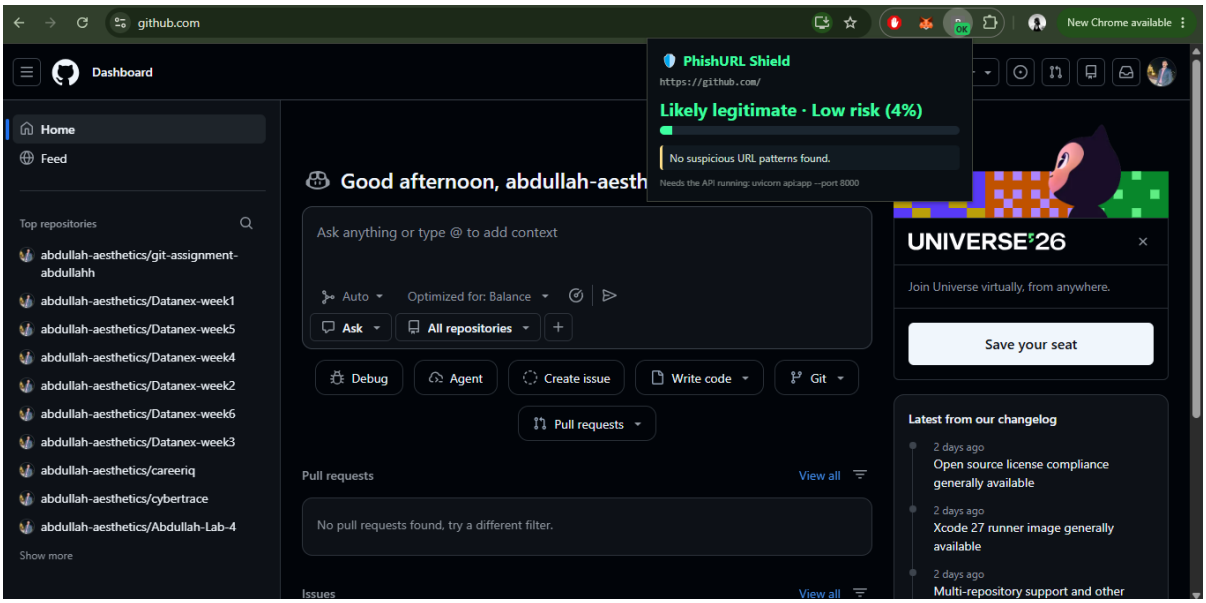


Figure 11: Extension on github.com: likely legitimate, low risk (4%), green badge

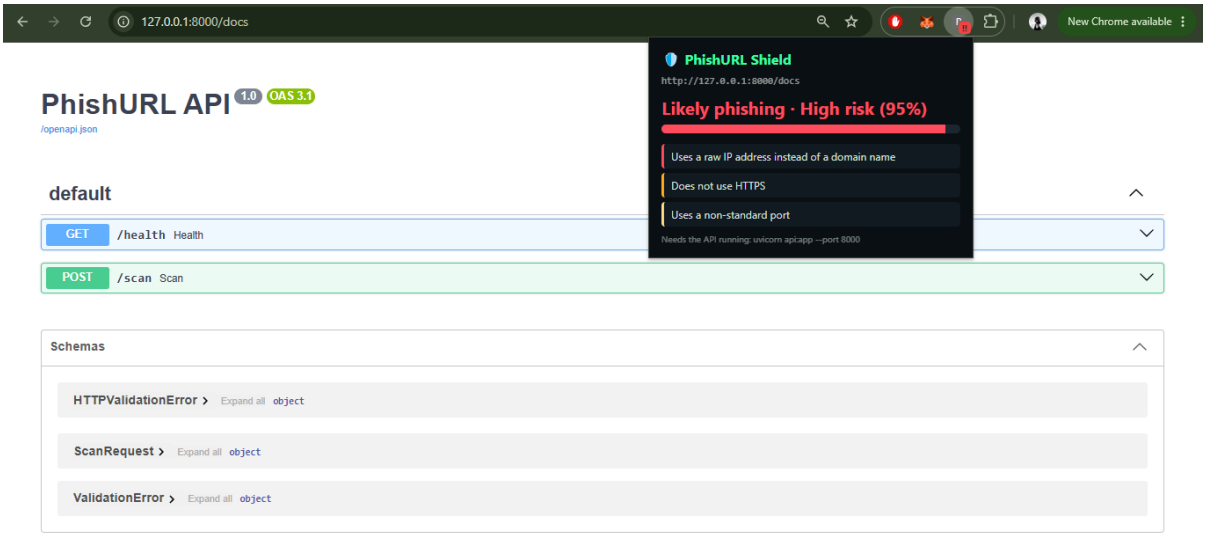


Figure 12: Extension on http://127.0.0.1:8000/docs: high risk (95%) because of a raw IP address, no HTTPS and a non-standard port

7. Dataset:-

The models were trained on the **Phishing Url** dataset ("A Dataset for Phishing Detection") by Hemanth Pingali, published on Kaggle (https://www.kaggle.com/datasets/hemanthpingali/phishing-url) under the CC BY-NC-SA 4.0 licence, which allows non-commercial, share-alike use such as this coursework. It is distributed as two Parquet files with 89 columns each: the URL, 87 pre-computed features and a status label.

Item	Value
Training.parquet	7,658 rows
Testing.parquet	3,772 rows
Combined, after removing 1 duplicate URL	11,429 URLs
Legitimate / phishing	5,715 / 5,714 (balanced)
Training / test split used	9,143 / 2,286 (stratified 80/20)

Two notes on the data. First, the Kaggle description says the testing file has no labels, but the downloaded copy contained labels, so both files were combined and re-split with a fixed random seed. There was no overlap between the two files. Second, the page does not name the original source of the data. Its 87-feature schema is consistent with the benchmark described by Hannousse and Yahiouche (2021), but this provenance is unverified.

Only the url and the label are used. The other 87 columns include page-content, WHOIS, traffic and Google-index features that require visiting the page or querying external services, so they cannot be computed for a new link in a live scanner that never opens the URL. PhishURL therefore computes its own features from the URL text.

8. Testing and Results:

8.1 Evaluation method:-

Each model was evaluated with 5-fold stratified cross-validation on the 9,143 training URLs and then once on the 2,286 untouched test URLs. The text vectoriser sits inside each scikit-learn pipeline, so it is refitted on the training folds only and no information leaks from the validation folds. The default model was chosen by cross-validated F1.

8.2 Results on the hold-out test set:-

Metric	Random Forest (21 features)	Char n-gram + LR	Hybrid (n-gram + features)
Accuracy	0.851	0.935	0.934
Precision	0.864	0.939	0.947
Recall	0.834	0.931	0.919
F1-score	0.849	0.935	0.933
ROC-AUC	0.926	0.982	0.984
CV F1 (mean ± std)	0.848 ± 0.006	0.932 ± 0.006	0.932 ± 0.005

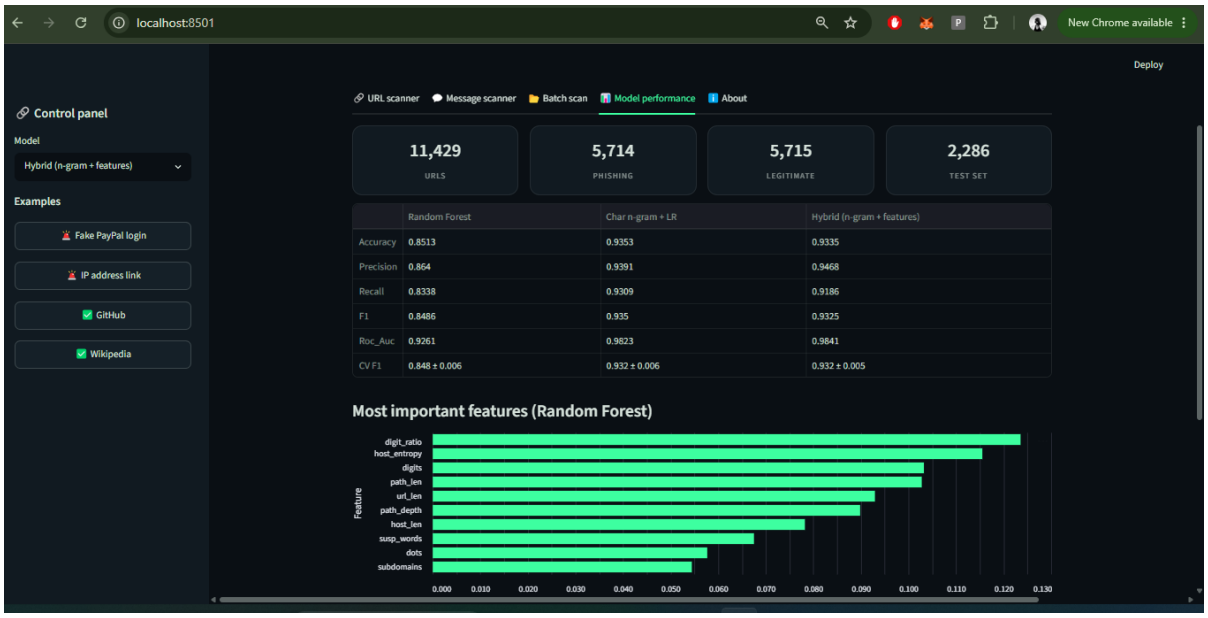


Figure 13: Model performance tab: dataset summary, metrics table and Random Forest feature importances

8.3 Confusion matrices:-

Model	Legitimate correct	Legitimate flagged (false alarm)	Phishing missed	Phishing caught
Random Forest	993	150	190	953
Char n-gram + LR	1,074	69	79	1,064
Hybrid	1,084	59	93	1,050

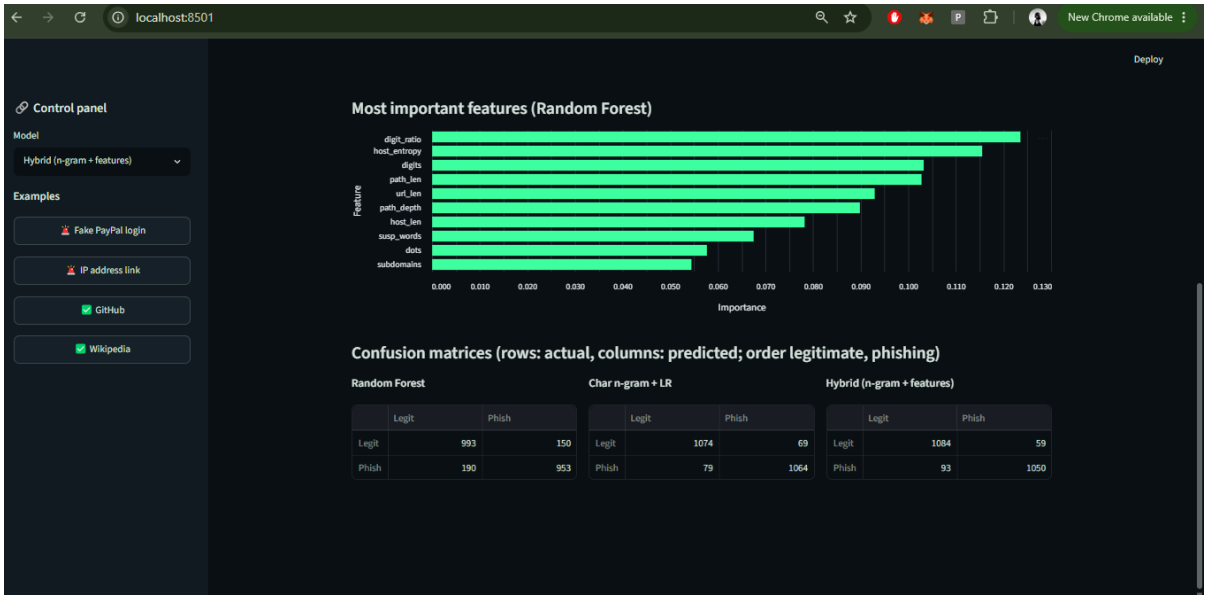


Figure 14: Confusion matrices of the three models on the 2,286 test URLs

8.4 Discussion:-

The two character n-gram models reach about 93.5% accuracy, far above the 85% of the Random Forest that uses only the 21 hand-built features. Reading the URL string directly captures patterns that the hand-built features miss. The Hybrid model was selected as the default because it ties the n-gram model on cross-validated F1 (0.932) and has the highest precision (0.947, with the fewest false alarms: 59) and the highest AUC (0.984). The n-gram model catches slightly more phishing URLs (recall 0.931, 79 missed against 93). The difference between the two is within one standard deviation of the cross-validation scores, so neither is clearly better. Because a missed phishing link is usually costlier than a false alarm, a deployment could lower the decision threshold to favour recall.

The most important features of the Random Forest are the digit ratio, the entropy of the host, the number of digits and the lengths of the URL and path. They suggest that machine-generated or obfuscated URLs are a strong phishing signal.

8.5 Supplementary analysis:-

Two additional checks were run with a separate one-off script that is not part of the repository. First, about 33% of the test URLs come from websites that also occur in the training data, which flatters any model that learns domain names. On the 64% of test URLs whose host never appeared in training, the n-gram models scored about 91% accuracy and the Random Forest about 80%. The figure of roughly 91% is therefore a more realistic estimate for new websites than 93.5%. Second, a Random Forest trained on all 87 pre-computed features of the dataset reached 96.3% accuracy on the original split, but those features need the page content and external lookups, which a live URL scanner does not have.

8.6 Manual testing of the deployed system:-

Input	Expected	Result
paypal-secure-login.top/verify/account.php?id=88231&session=4412	Phishing	High, 100%
192.168.44.12/signin/update.php?id=991	Phishing	High, 97.5%
bit.ly/3xYz45 (shortener test link)	Suspicious	High, 90%
github.com	Legitimate	Low, 5.3%
en.wikipedia.org/wiki/Phishing	Legitimate	Low, 0.2%
www.bbc.co.uk/news	Legitimate	Low, 1.3%
www.tapmad.com	Legitimate	Low, 0.6%
printify.com/app/auth/login	Legitimate	**High, 75.7% (false positive)**
mail.google.com/mail/u/0/#inbox	Legitimate	**High, 78.5% (false positive)**
Parcel scam SMS with link dhl-track-parcel.xyz	Phishing	High, 99%

Eight of ten manual tests were correct. The two failures are genuine services whose URLs contain words such as login, auth and inbox, which the model has learned to associate with phishing.

8.7 Software tests:-

Two automated pytest tests cover URL feature extraction and reasons, and model training and scoring (a phishing-style URL must score higher than a legitimate one). Both pass (2 passed).

9. Challenges:

- **Weak first model:** the original 21 hand-built features gave only about 85% accuracy on real data. Adding character n-gram TF-IDF raised it to about 93.5%.
- **Features that cannot be used live:** the dataset's strongest features (page content, WHOIS, Google index) need the page itself. The project had to rely on the URL text only.
- **False positives on login pages:** legitimate login and inbox URLs are flagged because their words resemble phishing patterns (Section 8.6).
- **Domain overlap inflates accuracy:** the same websites appear in training and test data, so a separate analysis was needed to estimate performance on unseen sites.
- **Data preparation:** the dataset arrived as Parquet files with text labels and one duplicate, and had to be converted into a clean binary format.
- **Reproducibility and deployment:** library versions had to be pinned so the deployed model loads without warnings, and the Python command-line tools were not on the system path, so commands were run through python -m.
- **GitHub accounts:** the repository and the Streamlit deployment had to be linked to the same GitHub account.
- **Extension hosting:** the extension talks to a local API, so it works on the author's machine and in a recorded demonstration rather than as a hosted service.

Limitations:-

PhishURL judges the URL text only. It does not look at the page, the domain's age or reputation, redirects, or blocklists, and it can be fooled by a phishing link that looks ordinary or flag a genuine link that looks unusual. Accuracy on new websites is closer to 91% than 93.5%. The system should be used as decision support and not as the only defence.

10. Future Improvements:

- Add an allow-list of well-known domains and a domain-reputation lookup to cut false positives such as mail.google.com.
- Add domain age and WHOIS data, blocklist checks (for example PhishTank or OpenPhish) and redirect resolution.
- Train a text classifier for SMS and WhatsApp scam wording (for example on the UCI SMS Spam Collection) and combine it with the link score.
- Compare against transformer models such as DistilBERT trained in Google Colab.
- Evaluate with a split grouped by domain, and test on a second independent dataset.
- Host the API online so the extension works without a local server, and publish the extension.
- Show which characters or n-grams drove each decision, for example with Logistic Regression coefficients or SHAP.

11. Conclusion:

PhishURL shows that phishing links can be detected from the URL text alone with about 93.5% accuracy on a held-out test set (about 91% on websites not seen during training), and that the decision can be explained to the user. The project compares three models, selects the default by cross-validation, and delivers the result as a deployed dashboard, a REST API and a Chrome extension, together with

automated tests and public source code. Its honest limitations are false alarms on genuine login pages and the lack of page-level and reputation signals. These can be reduced with the improvements listed above.

12. References:

1. Pingali, H. Phishing Url: A Dataset for Phishing Detection. Kaggle. https://www.kaggle.com/datasets/hemanthpingali/phishing-url (licence CC BY-NC-SA 4.0).
2. Hannousse, A. and Yahiouche, S. (2021). Towards benchmark datasets for machine learning based website phishing detection: An experimental study. Engineering Applications of Artificial Intelligence, 104, 104347. (Possible origin of the dataset schema; unverified.)
3. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.
4. Streamlit documentation. https://docs.streamlit.io
5. FastAPI documentation. https://fastapi.tiangolo.com
6. Chrome Extensions (Manifest V3) documentation. https://developer.chrome.com/docs/extensions
7. Anti-Phishing Working Group (APWG). Phishing Activity Trends Reports. https://apwg.org
8. PhishURL source code. https://github.com/abdullah-aesthetics/PhishURL